

Algoritmer og Datastrukturer

28. september 2026

Prioritetskøer

Opgaver i tidskompleksitet (1)

Hvad er Store O tidskompleksiteten af nedenstående metode. Begrund dit svar.

```
public static int myMethod( int[] arr )
{
    int x = 0;
    for (int i = 0; i < arr.length; i++)
        for (int j = 0; j < arr.length/2; j++)
            for (int k = 0; k < arr.length; k++)
                {
                    x++;
                    if (k==1)
                        break;
                }
    return x;
}
```

Opgaver i tidskompleksitet (2)

Hvad er Store O tidskompleksiteten af nedenstående metode. Begrund dit svar.

```
public static int myMethod1( int[] arr )
{
    int x = 0;
    for (int i = 0; i < arr.length/2; i++)
        for (int j = 0; j < arr.length; j++)
            for (int k = 0; k < arr.length; k++)
            {
                x++;
                if (k==arr.length/2)
                    break;
            }
    return x;
}
```

The attribute *length* denotes the number of elements in the array.

Opgaver i tidskompleksitet (3)

Hvad er Store O tidskompleksiteten af metoden `func1`. Begrund dit svar.

```
public static int func2(int N)
{
    int res = 0;
    for (int i = 0; i < N; i++)
        res = res + 1;
    return res;
}
```

```
public static int func1(int N)
{
    int x = 0;
    for (int i = 0; i < N; i++)
        x = x + func2(N);
    return x;
}
```

Øvelser – kan løses i hånden

1

En hashtabel har plads til 16 elementer.

Indsæt fem elementer, som alle hasher til samme position i tabellen med quadratic probing.

2

En hashtabel har plads til 11 elementer.

Indsæt seks elementer, som alle hasher til samme position i tabellen med quadratic probing.

3

Hvordan skal man håndtere *sletninger*, hvis der anvendes probing/open addressing?

Indekserne 1, 2, 3, 4, 7, 8 og 10 er ledige. Bekræft at tabellen er korrekt og ret eventuelle fejl. Vis hvordan tabellen ser ud, efter at elementerne 1, 12, 23, din alder og dit eksamensnummer er blevet indsat

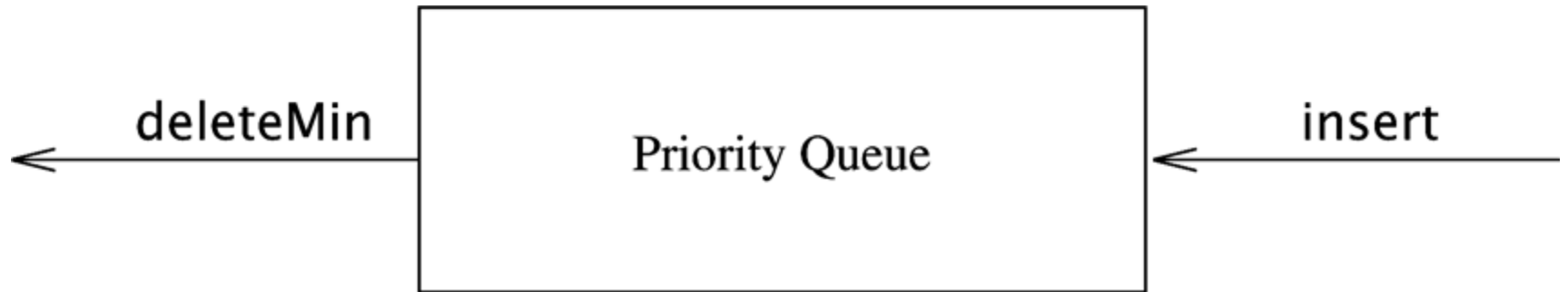
Indeks Værdi

0	22
1	
2	
3	
4	
5	5
6	16
7	
8	
9	27
10	

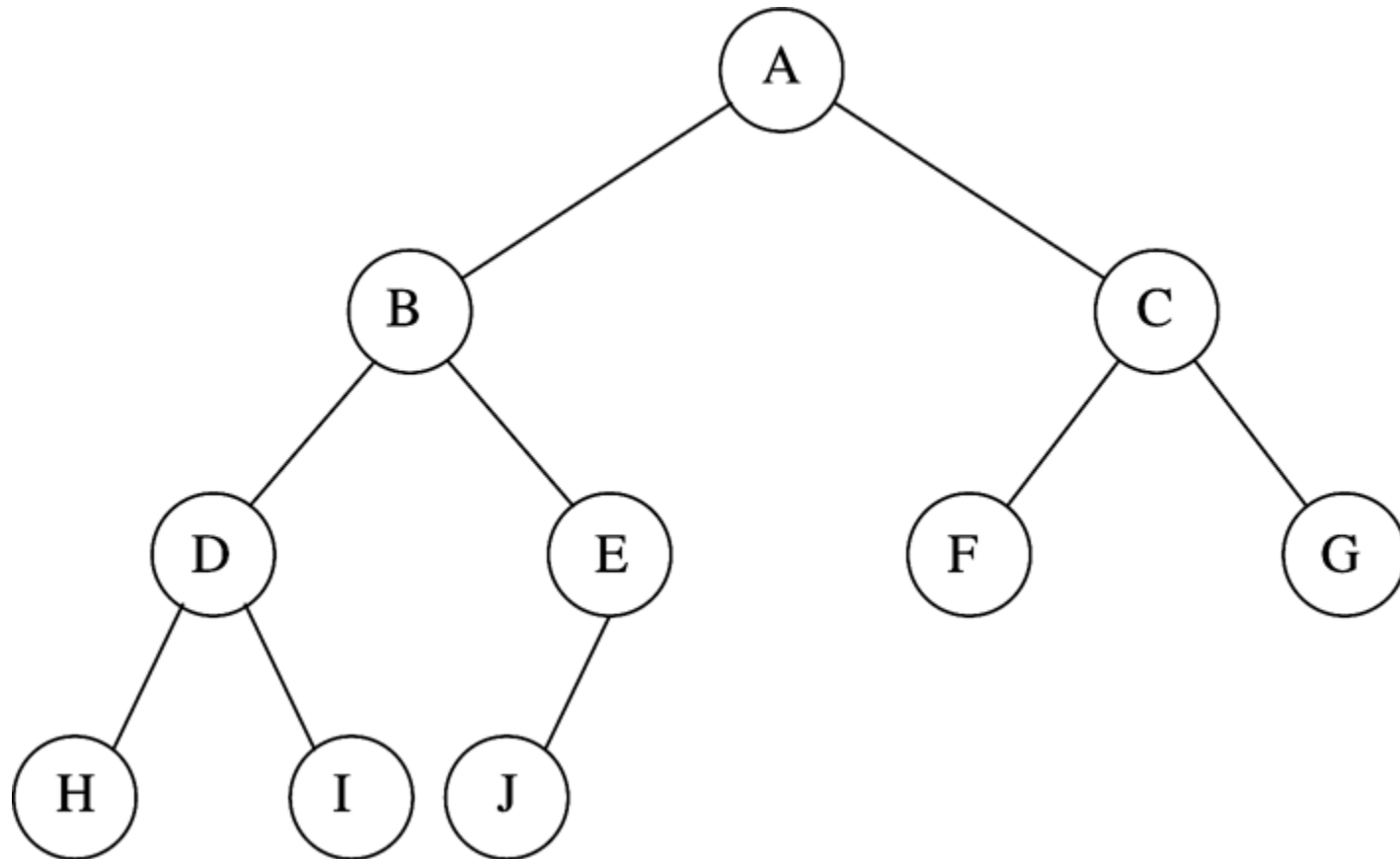
Prioritetskø

- Baggrund: behovet for at kunne afvikle køer efter et andet princip end FIFO.
- Eksempler: printjobs, CPU-schedulering, hasteordre.
- Nødvendige operationer: `insert` og `deleteMin`.
- Elegant ADT kan tilsikre `insert` with $O(\log N)$ i modsætning til forventet $O(N)$.

Prioritetskø



Forudsætninger – et par ord om træer og *binære* træer i særdeleshed



Begreber (suppl. til forrige slide)

- tree – binære og andre; **rekursiv** datastruktur
- node – element med data
- edge – forbindelse mellem noder
- root – indgangspunktet til træet
- child – nul, et, to eller flere (0..m)
- parent – roden er forældreløs
- leaves – noder uden børn

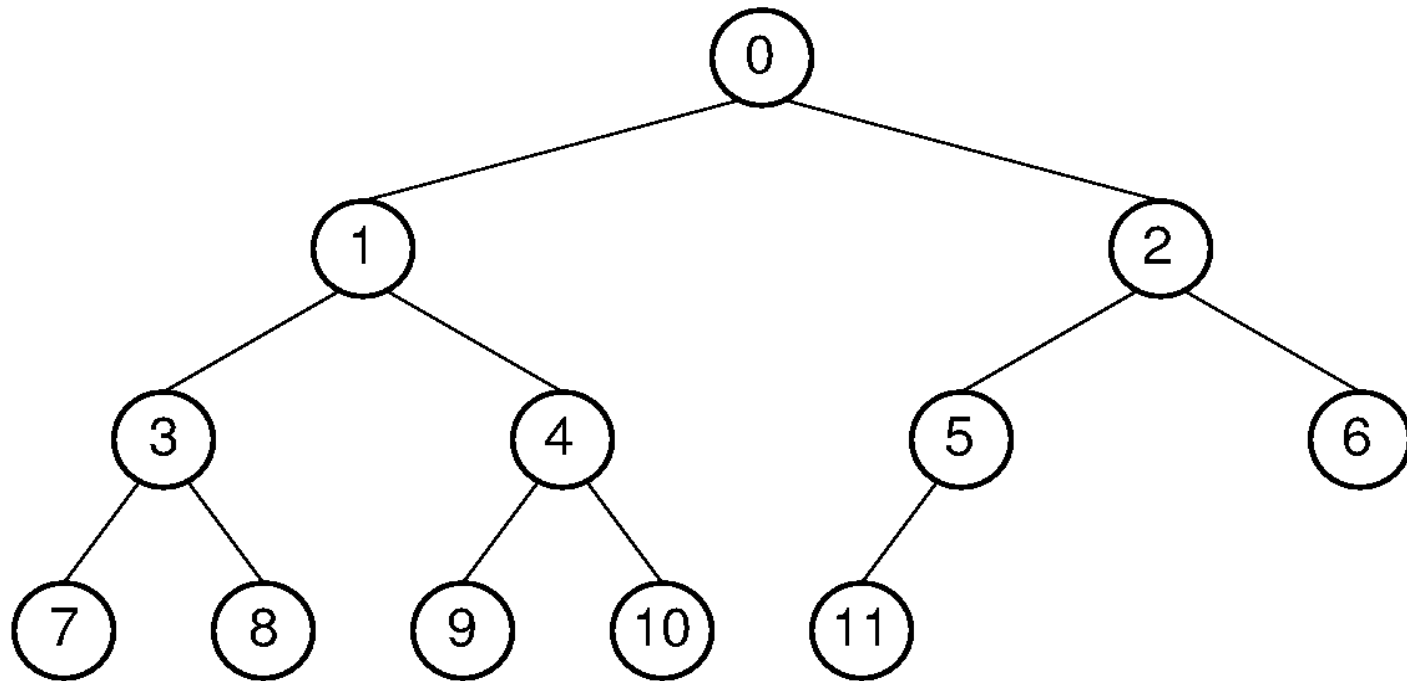
Begreber

- depth – afstand fra node til root
- height – længste distance fra node til leaf (blad)
- sibling, grandparent, grandchild – sædvanlige familierelationer

Binary trees kan være.....

- Komplette
- Perfekte
- Balancerede

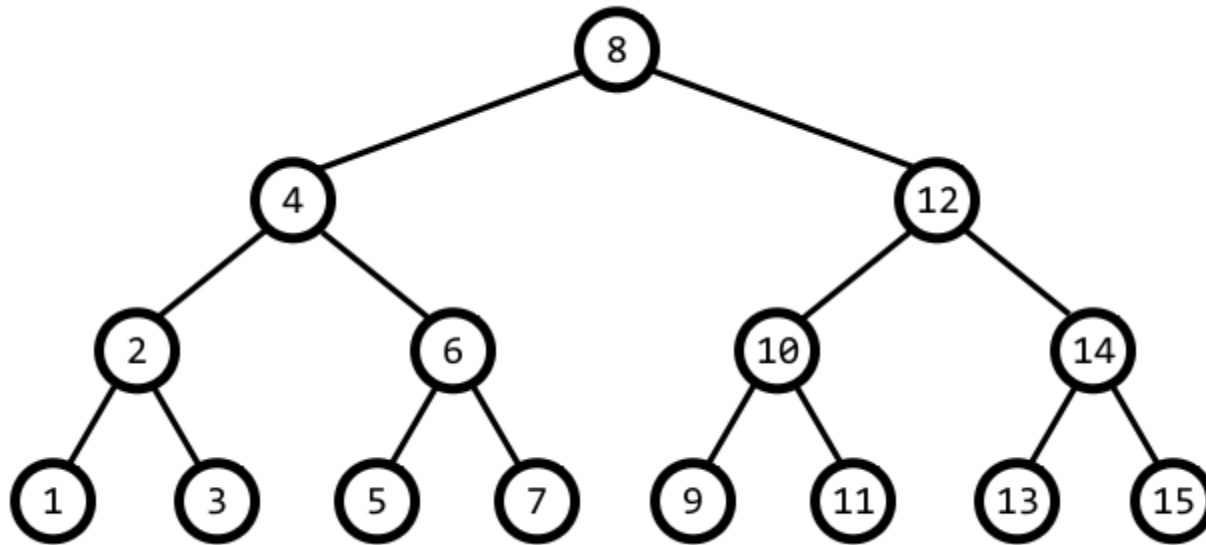
Komplet binært træ



(a)

Alle niveauer er fyldt op fra venstre mod højre.
Hvis node 6 eller node 7 fjernes, er træet ikke længere komplet.

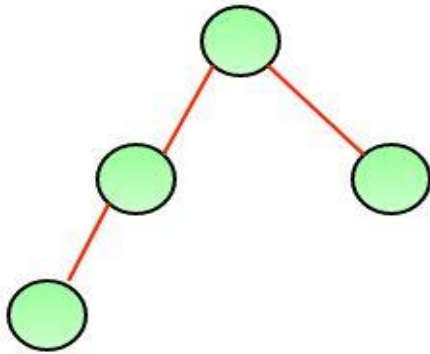
Perfekt binært træ



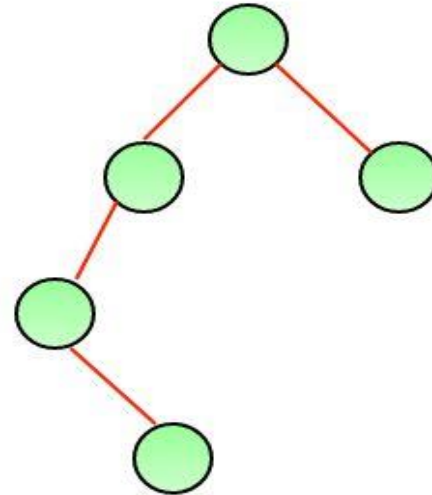
Alle niveauer er fyldt fra venstre til højre.

$$N = 2^{h+1} - 1$$

Balanceret binært træ



A height balanced tree



Not a height balanced tree

Højdeforskellen mellem subtræer må ikke være større end 1 (en).

Binært Træ

```
class BinaryNode
{
    Object      element;      // The data in the node
    BinaryNode left;          // Left child
    BinaryNode right;         // Right child
}
```

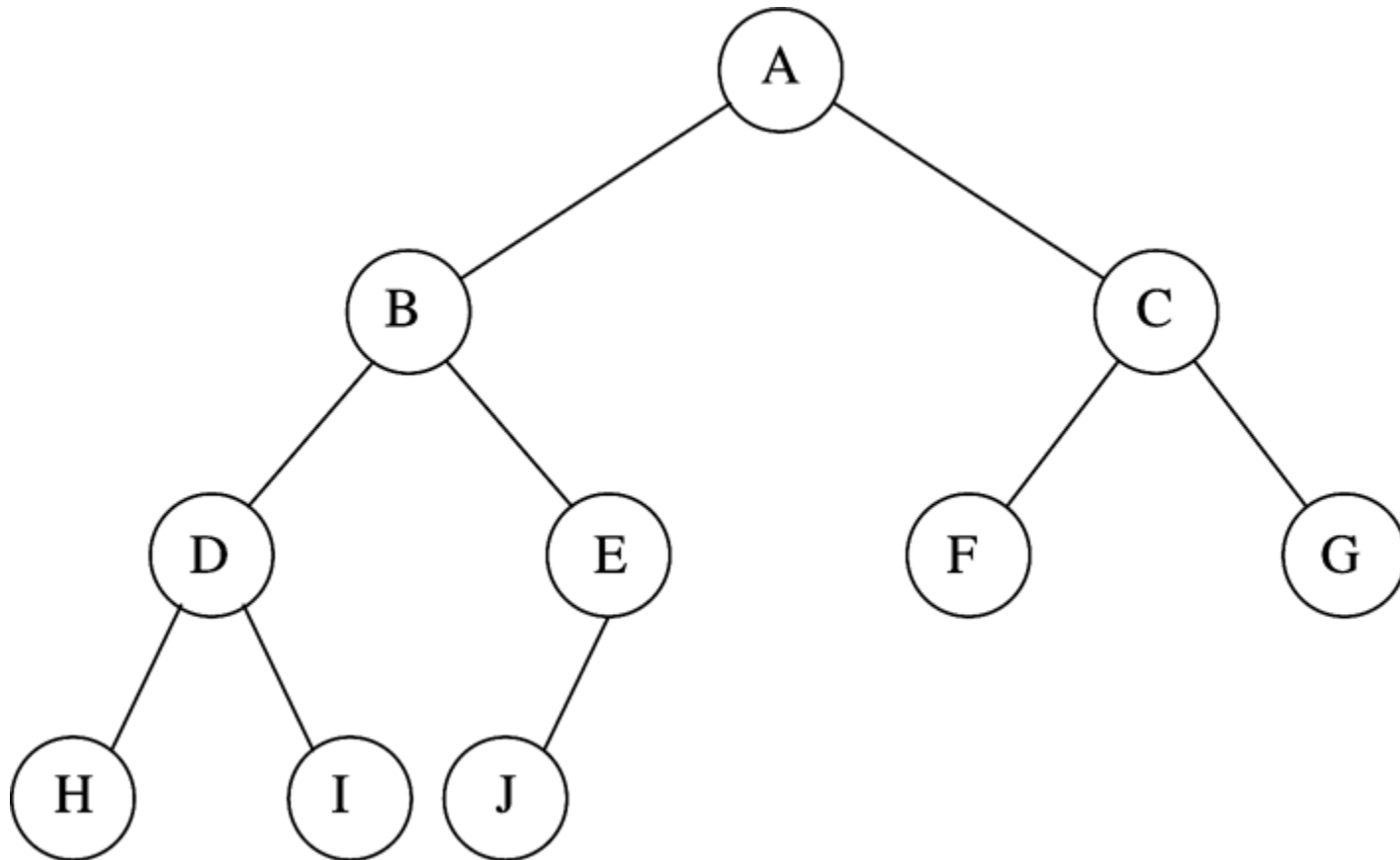
Der behøver kun at være ét dataelement i klassen

BinaryTree

Hvilket?

Prioritetskø – komplet binært træ

køen vokser fra venstre mod højre på laveste niveau



Implementering

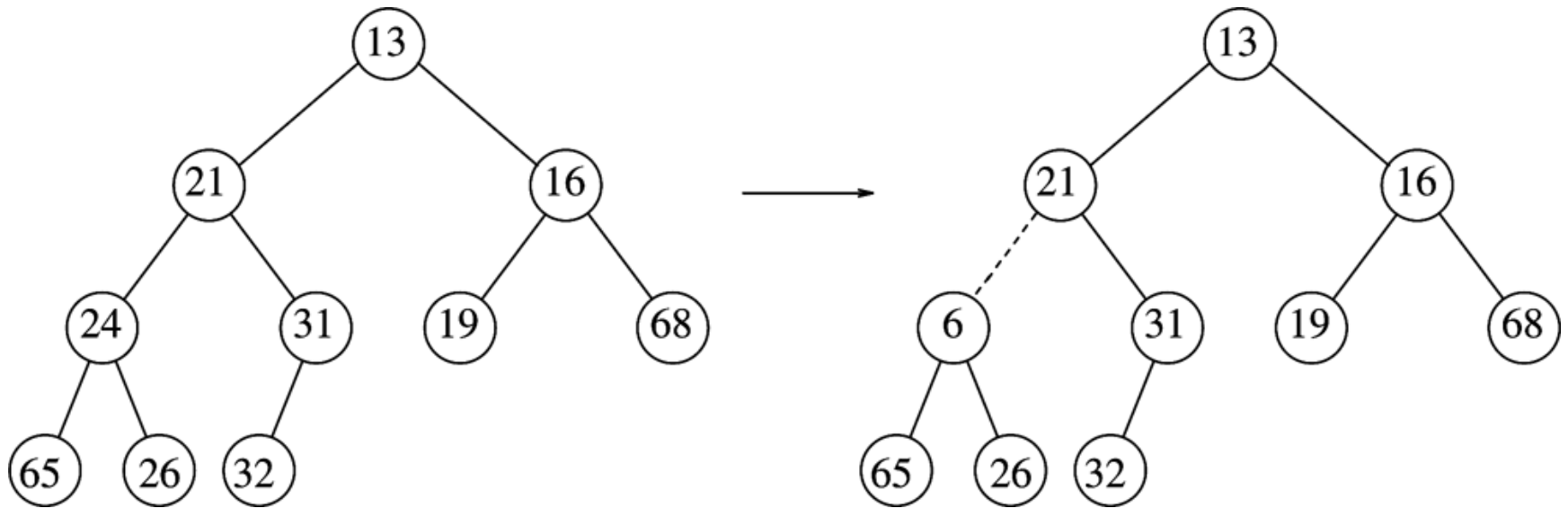
Almindeligt array hvor indeks 0 ikke benyttes

	A	B	C	D	E	F	G	H	I	J			
0	1	2	3	4	5	6	7	8	9	10	11	12	13

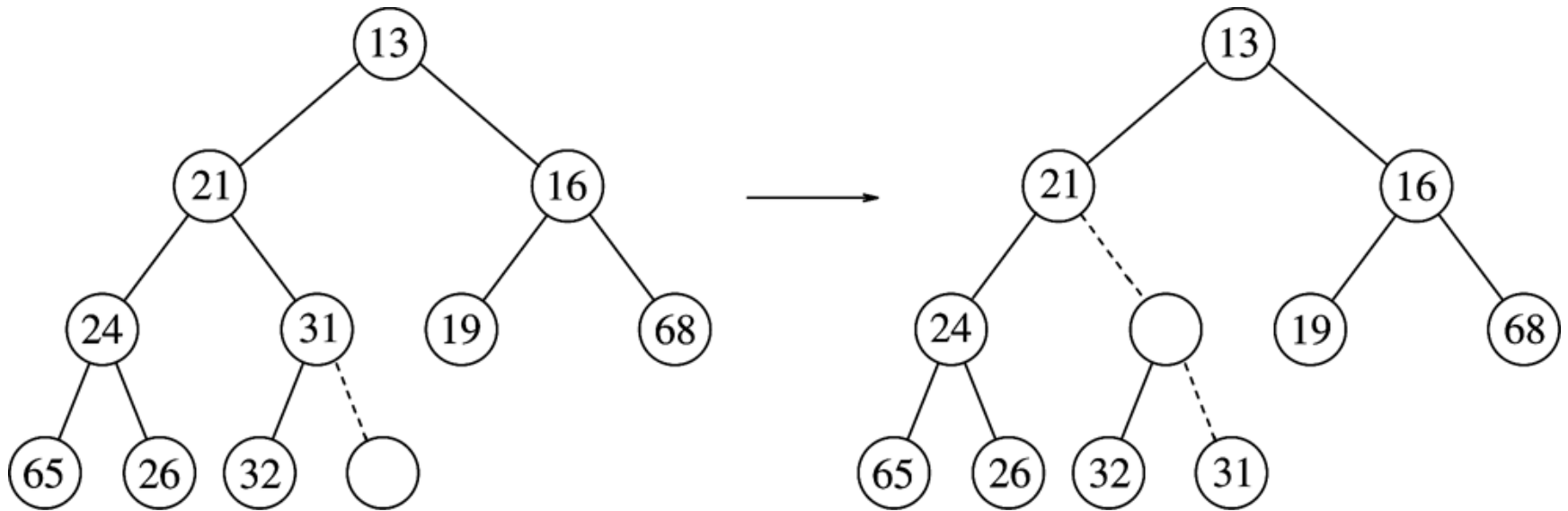
Heap order property

- Roden indeholder det mindste element.
- Enhver node er mindre end alle dens efterkommere.
- En prioritetskø kaldes sommetider for en binary *heap*.
- Dette må ikke forveksles med *heapen* i C++ runtime environment.

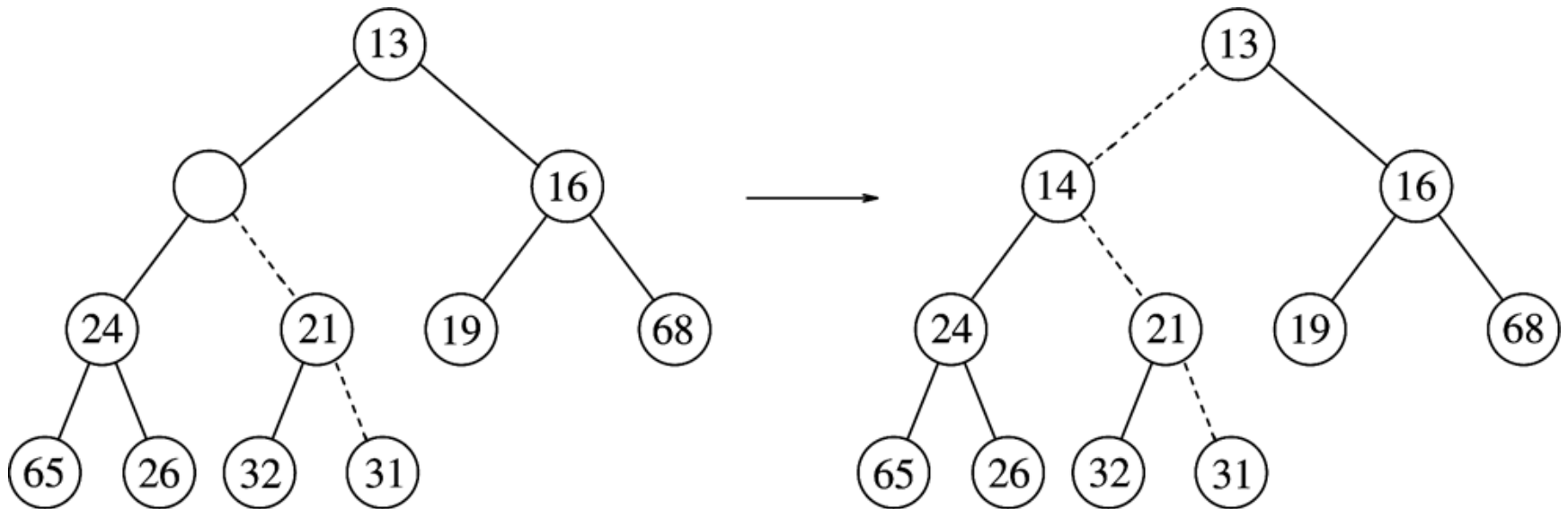
Komplette binære træer; heaps?



Insert 14 – step 1 & 2



Insert 14 – step 3 & 4

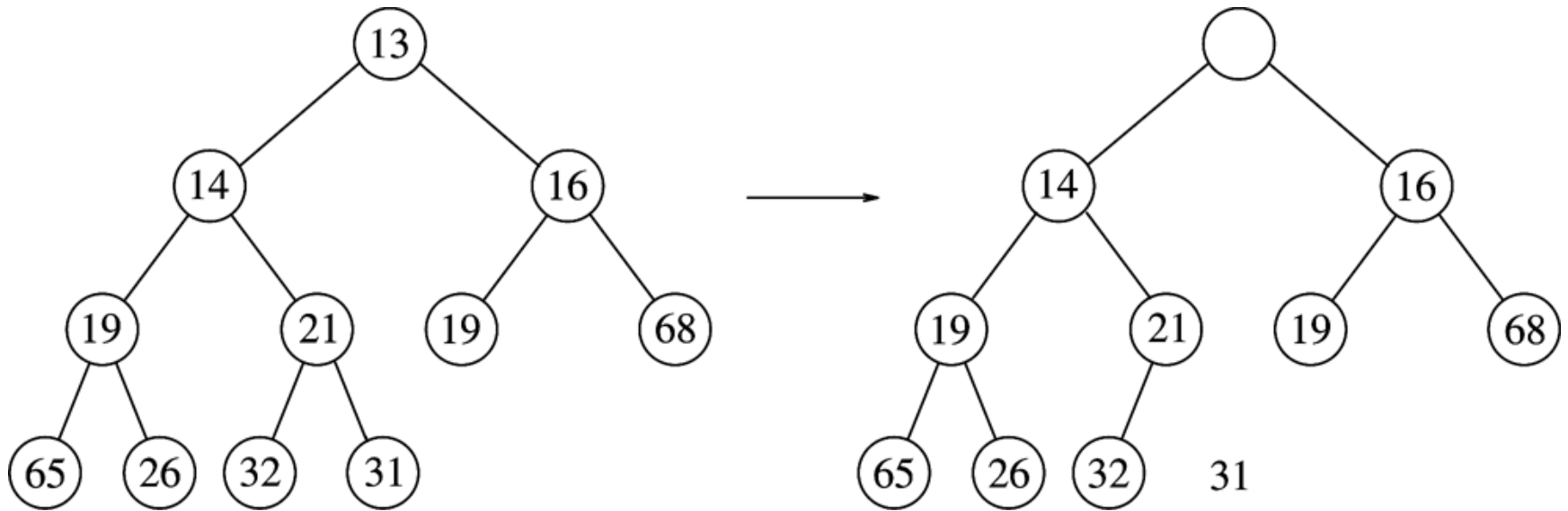


Insert() - Java

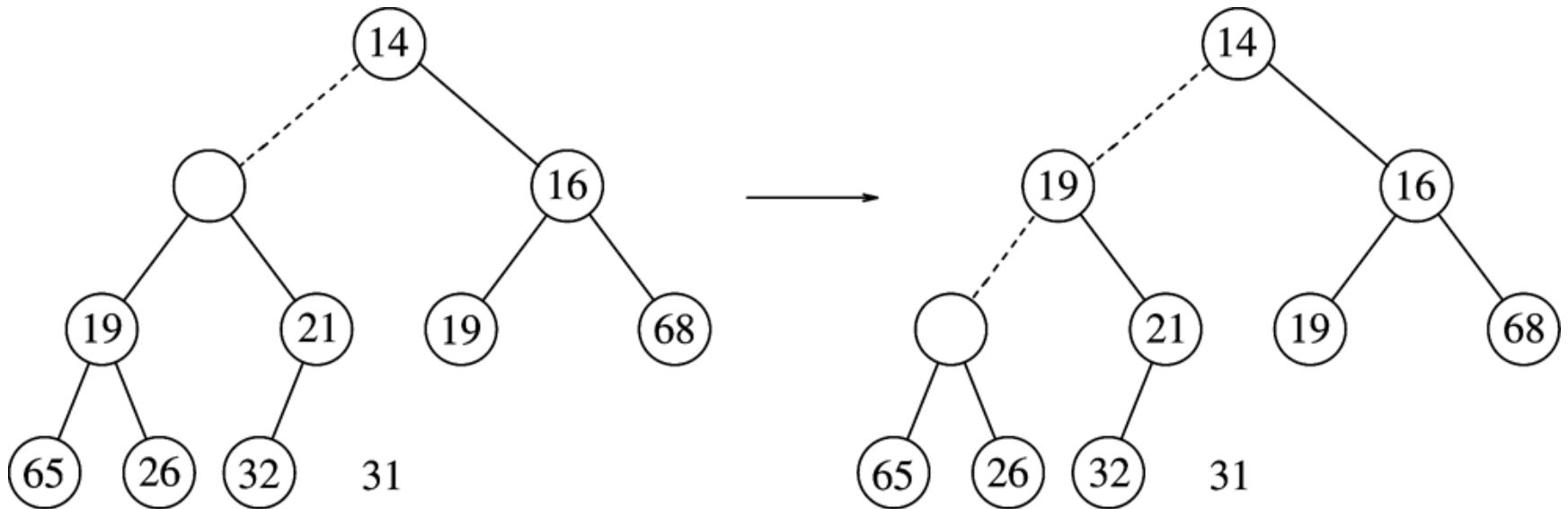
```
public void insert( AnyType x )
{
    if( currentSize == array.length - 1 )
        enlargeArray( array.length * 2 + 1 );

    // Percolate up
    int hole = ++currentSize;
    for( ; hole > 1 && x.compareTo( array[ hole / 2 ] ) < 0; hole /= 2 )
        array[ hole ] = array[ hole / 2 ];
    array[ hole ] = x;
}
```

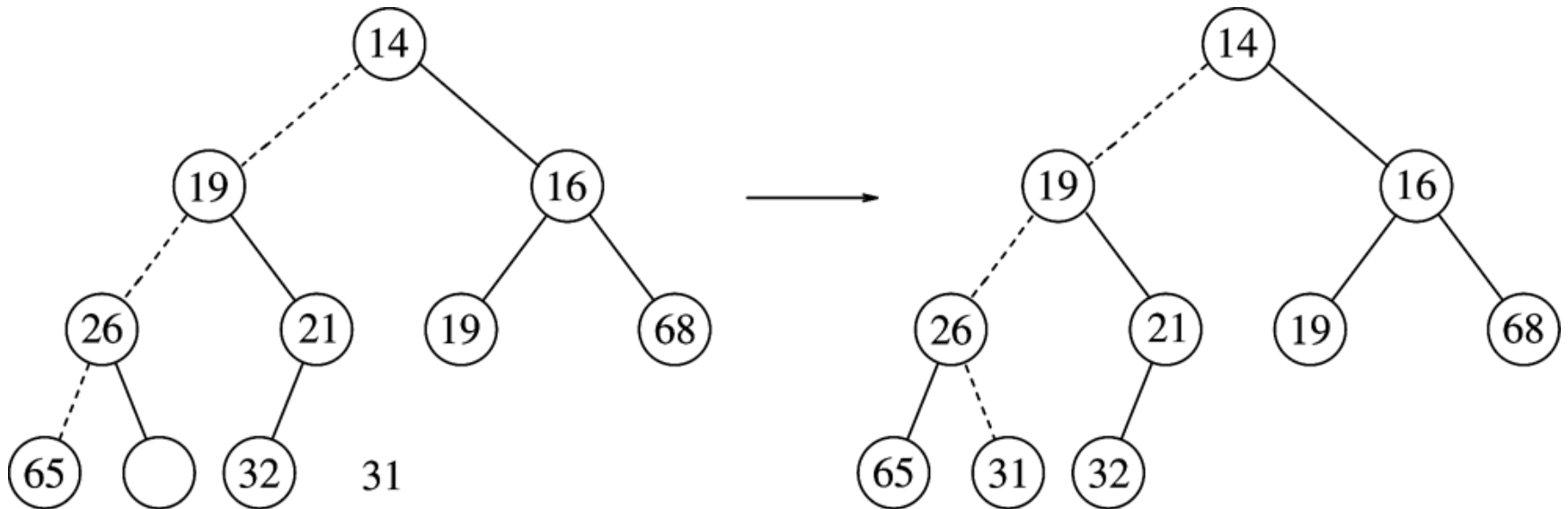
DeleteMin – step 1 & 2



DeleteMin – step 3 & 4



DeleteMin – step 5 & 6



deleteMin() – Java

```
public AnyType deleteMin()  
{  
    if (isEmpty())  
        throw new UnderflowException();  
  
    AnyType minItem = findMin();  
    array[1] = array[currentSize--];  
    percolateDown(1);  
  
    return minItem;  
}
```

percolateDown ()

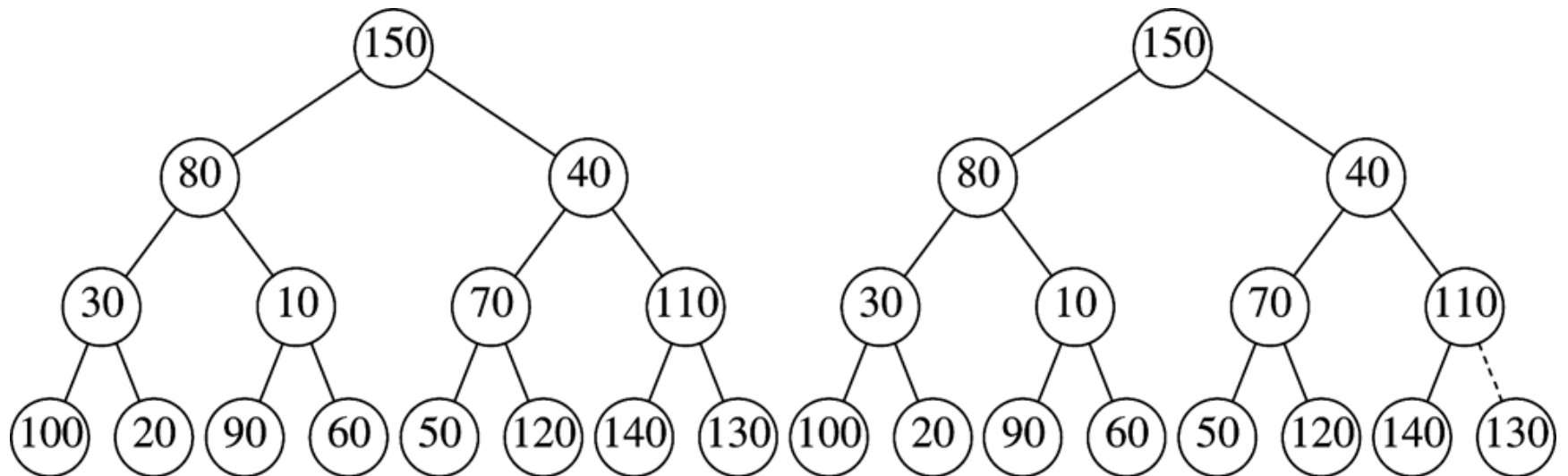
```
private void percolateDown( int hole )
{
    int child;
    AnyType tmp = array[ hole ];

    for( ; hole * 2 <= currentSize; hole = child )
    {
        child = hole * 2;
        if( child != currentSize &&
            array[ child + 1 ].compareTo( array[ child ] ) < 0 )
            child++;
        if( array[ child ].compareTo( tmp ) < 0 )
            array[ hole ] = array[ child ];
        else
            break;
    }
    array[ hole ] = tmp;
}
```

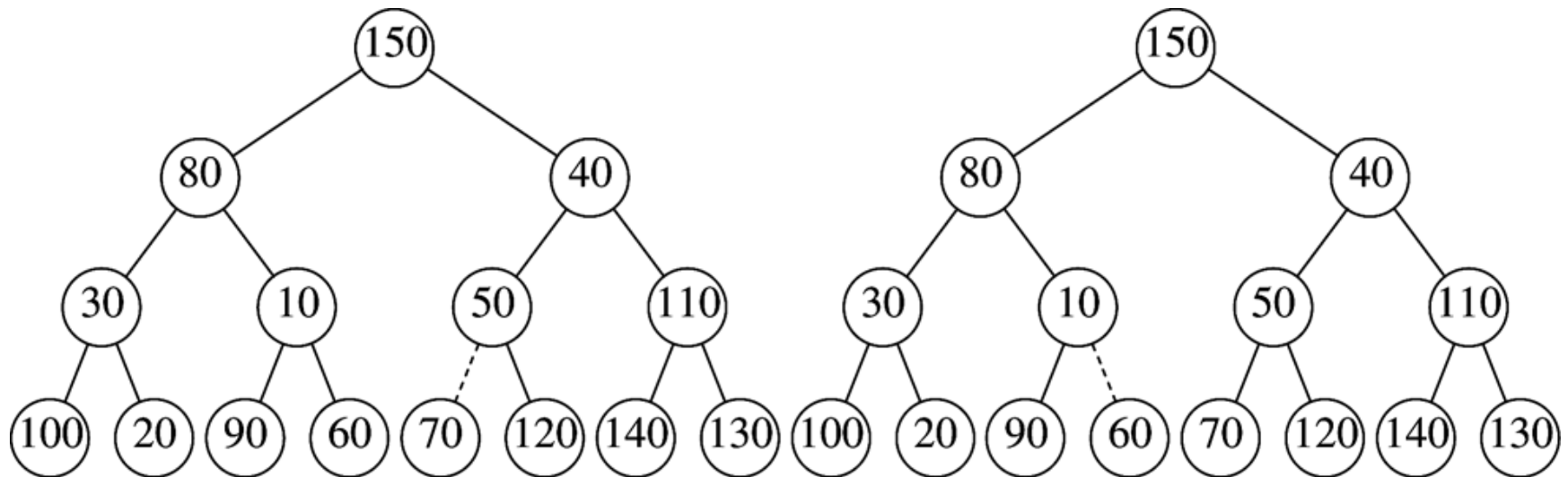
buildHeap ()

- Lave et binært træ (array minus indeks 0) af samtlige indgående elementer.
- Kald `percolateDown ( )` for all noder under roden. Begynd med *rightmost deepest* som ikke er et blad.

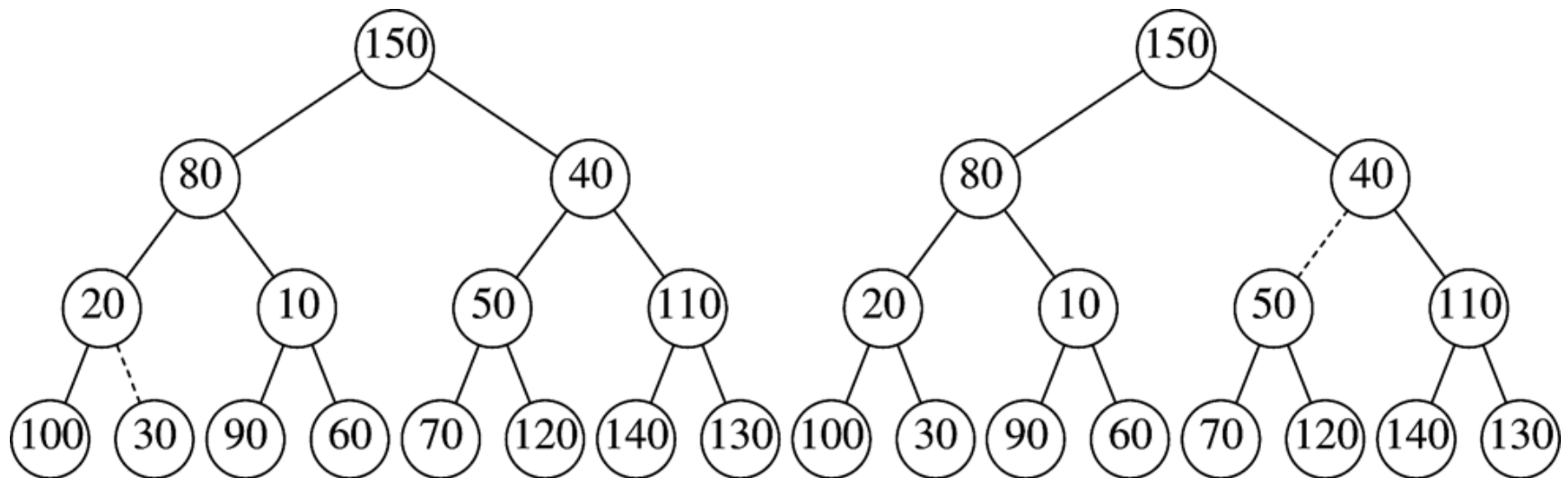
percolateDown (7)



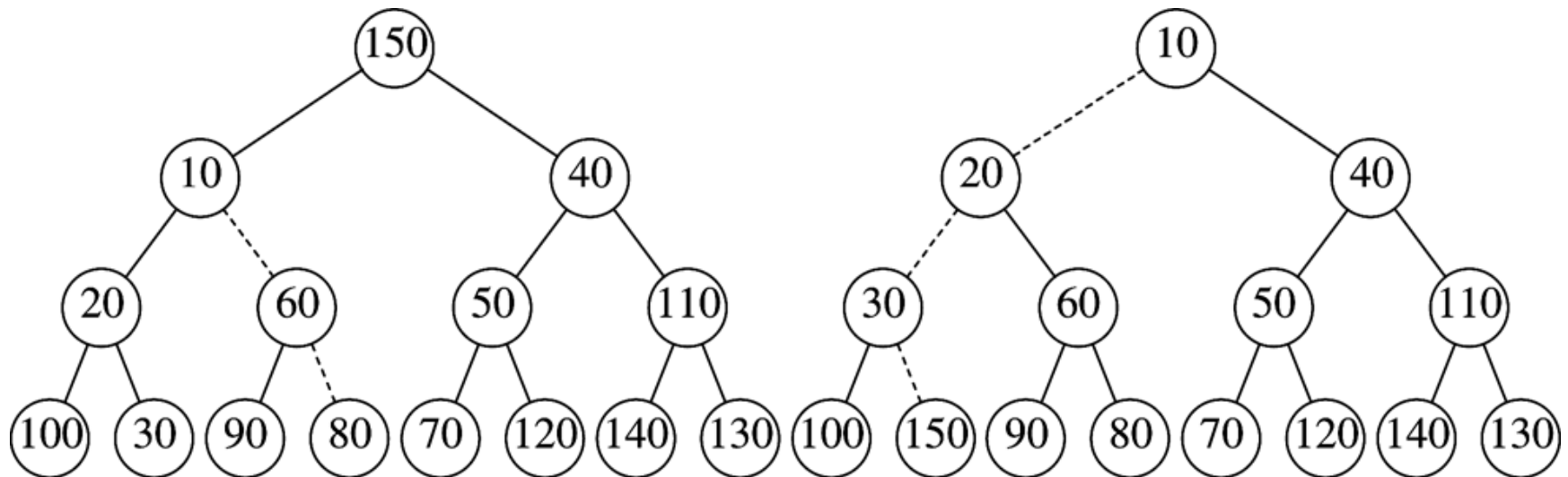
percolateDown(6) & (5)



percolateDown(4) & (3)



percolateDown(2) & (1)



buildHeap ()

- Påstand: `buildHeap ( )` is $O(N)$.
- Dette kan bevises ved at finde den øvre grænse for stiplede linjer i de foregående figurer.
- Som må være lig med summen af højderne af samtlige træets noder.
- Et perfekt binært træ med højden h har $2^{h+1} - 1$ noder, hvilket medfører at $N = 2^{h+1} - 1$
- Bevis at summen af nodernes højder er $2^{h+1} - 1 - (h+1)$
- Derfor er summen af højderne og antallet af noder lineært forbundne.
- Summen af højderne er mindre end N .

Bevis

$$S = \sum_{i=0}^h 2^i (h - i)$$

$$= h + 2(h-1) + 4(h-2) + 8(h-3) + \dots + 2^{h-1}(1)$$

$$2S = 2h + 4(h-1) + 8(h-2) + 16(h-3) + \dots + 2^h(1)$$

$$S = -h + 2 + 4 + 8 + \dots + 2^{h-1} + 2^h$$

$$S = 2^{h+1} - 1 - (h+1)$$

Hvorfor er *buildHeap()* vigtig?

- Sortering er en meget vigtig øvelse.
- Trivielle sorteringsalgoritmer har kvadratisk tidskompleksitet.
- Med *buildHeap()* kan man opnå $O(N \log N)$.
- Først bygger man en heap (prioritetskø) af sine usorterede tal. Det kræver max. N operationer.
- Så kaldes *deleteMin()* på alle elementer. Det tager $N \log N$ operations. Og tallene er sorterede.
- $N + N \log(N) \rightarrow O(N \log N)$.

Trivial sorteringsalgoritme - $O(N^2)$

Bubblesort:

```
1  public void bubbleSort(int array[]) // Java
2  {
3      for (int i = 0; i < array.length - 1; i++)
4      {
5          boolean swapped = false;
6          for (int j = 0; j < array.length - i - 1; j++)
7          {
8              if (array[j] > array[j+1])
9              {
10                 swap(array,j,j+1);
11                 swapped = true;
12             }
13         }
14         if (!swapped)
15             return;
16     }
17 }
```

Forskel på $O(N^2)$ og $O(N \log N)$

- $N = 65.536$
- $N^2 = 4.294.967.296$
- $N \log N = 1.048.576$

Vigtigste slides

7

13-14

17-19

34-36

Eksamensopgave – 5 % (9 min.)

Den nedenstående tabel kan *ikke* repræsentere en prioritetskø, hvor enhver node skal være mindre end alle sine efterkommere.

Denne såkaldte *heap-order* fejler ét sted. I hvilket indeks?

I øverste række er prioriteterne angivet; i nederste række står de tilsvarende tabelindeks.

	14	17	16	28	22	65	29	31	30	26	23	89	64	35	32	48	47	46	45
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Øvelser

på *itslearning*